

Coarsened Tiled Matmul — Complete Visual Hand-Trace

PMPP Chapter 6.3 · every load, sync, compute, and write · numbers generated by simulating the exact kernel index formulas

Setup

N	TILE_WIDTH	COARSE_FACTOR	gridDim	blockDim	phases	strips/thread
4	2	2	(1, 2)	(2, 2)	2	2

gridDim.x = $N / (\text{TILE_WIDTH} \times \text{COARSE_FACTOR}) = 1$ — one block covers ALL four output columns (two strips of two). gridDim.y = $N / \text{TILE_WIDTH} = 2$ — two blocks stacked vertically, Block(0,0) for output rows 0-1 and Block(0,1) for output rows 2-3. Each block has 4 threads.

Input matrices — decode before anything else

A is row-major: $A[\text{row}][\text{col}]$ lives at flat address $\text{row} \times N + \text{col}$. B's flat array is IDENTICAL to A's, but the kernel's corner-turned load ($B[\text{col} \times N + \text{row_part}]$) interprets it as COLUMN-major: $B[\text{row}][\text{col}]$ lives at flat address $\text{col} \times N + \text{row}$. Same 16 numbers, different decode, different logical matrix. This is the single easiest thing to get wrong when checking by hand.

A (row-major decode)

	col 0	col 1	col 2	col 3
row 0	1	2	3	4
row 1	5	6	7	8
row 2	9	10	11	12
row 3	13	14	15	16

B (column-major decode)

	col 0	col 1	col 2	col 3
row 0	1	5	9	13
row 1	2	6	10	14
row 2	3	7	11	15
row 3	4	8	12	16

Flat arrays, both = [1, 2, 3, ..., 16]. Notice B's logical matrix is the transpose of A's — that is what a column-major decode of row-major-ordered numbers produces.

Per-thread constants (computed once, before any loop)

block	thread (tx,ty)	row = by*2+ty	colStart = bx*4+tx	owned columns (c=0, c=1)
(0,0)	(0,0)	0	0	0, 2
(0,0)	(1,0)	0	1	1, 3
(0,0)	(0,1)	1	0	0, 2
(0,0)	(1,1)	1	1	1, 3
(0,1)	(0,0)	2	0	0, 2
(0,1)	(1,0)	2	1	1, 3
(0,1)	(0,1)	3	0	0, 2
(0,1)	(1,1)	3	1	1, 3

Read one row of this table as a contract: that thread will compute exactly the output elements $P[\text{row}][\text{colStart}]$ and $P[\text{row}][\text{colStart}+2]$, and nothing else, for the entire kernel.

Block (0,0) — output rows 0 and 1, all columns

Phase ph = 0

Mds load — happens ONCE this phase, outside the c loop. Each thread fetches $A[\text{row}][0..1 \text{ range}]$ via $A[\text{row} * N + (0 * 2 + \text{tx})]$:

thread	global read	value	written to
(0,0)	A[0]	1	Mds[0][0]
(1,0)	A[1]	2	Mds[0][1]
(0,1)	A[4]	5	Mds[1][0]
(1,1)	A[5]	6	Mds[1][1]

Resulting Mds tile (shared by the whole block, reused for BOTH strips below):

	col 0	col 1
row 0	1	2
row 1	5	6

Strip c = 0 (col = colStart + 0*2)

Nds load — inside the c loop, fresh load. Corner-turned: $Nds[\text{tx}][\text{ty}] = B[\text{loadCol} * N + (0 * 2 + \text{tx})]$, where $\text{loadCol} = \text{bx} * 4 + 0 * 2 + \text{ty}$ (ty, not tx — the load column must match the slot's second index):

thread	loadCol	global read	value	written to
(0,0)	0	B[0]	1	Nds[0][0]
(1,0)	0	B[1]	2	Nds[1][0]
(0,1)	1	B[4]	5	Nds[0][1]
(1,1)	1	B[5]	6	Nds[1][1]

Nds buffer now holds:

	col 0	col 1
row 0	1	5
row 1	2	6

__syncthreads() — every load above must land before any thread computes.

Compute — each thread updates ONLY Pvalue[0]: for k in 0..1, $Pvalue[0] += Mds[\text{ty}][k] * Nds[k][\text{tx}]$

thread	terms (k=0 + k=1)	adds	Pvalue[0] running total
(0,0)	$1 \times 1 + 2 \times 2$	5	5
(1,0)	$1 \times 5 + 2 \times 6$	17	17
(0,1)	$5 \times 1 + 6 \times 2$	17	17
(1,1)	$5 \times 5 + 6 \times 6$	61	61

__syncthreads() — every thread must finish READING Nds before the next strip (or next phase) overwrites it.

Strip c = 1 (col = colStart + 1*2)

Nds load — inside the c loop, OVERWRITES the c=0 contents — same buffer, new data. Corner-turned: $Nds[tx][ty] = B[loadCol*N + (0*2 + tx)]$, where $loadCol = bx*4 + 1*2 + ty$ (ty, not tx — the load column must match the slot's second index):

thread	loadCol	global read	value	written to
(0,0)	2	B[8]	9	Nds[0][0]
(1,0)	2	B[9]	10	Nds[1][0]
(0,1)	3	B[12]	13	Nds[0][1]
(1,1)	3	B[13]	14	Nds[1][1]

Nds buffer now holds:

	col 0	col 1
row 0	9	13
row 1	10	14

__syncthreads() — every load above must land before any thread computes.

Compute — each thread updates ONLY Pvalue[1]: for k in 0..1, $Pvalue[1] += Mds[ty][k] * Nds[k][tx]$

thread	terms (k=0 + k=1)	adds	Pvalue[1] running total
(0,0)	$1 \times 9 + 2 \times 10$	29	29
(1,0)	$1 \times 13 + 2 \times 14$	41	41
(0,1)	$5 \times 9 + 6 \times 10$	105	105
(1,1)	$5 \times 13 + 6 \times 14$	149	149

__syncthreads() — every thread must finish READING Nds before the next strip (or next phase) overwrites it.

Phase ph = 1

Mds load — happens ONCE this phase, outside the c loop. Each thread fetches $A[row][2..3 \text{ range}]$ via $A[row*N + (1*2 + tx)]$:

thread	global read	value	written to
(0,0)	A[2]	3	Mds[0][0]
(1,0)	A[3]	4	Mds[0][1]
(0,1)	A[6]	7	Mds[1][0]
(1,1)	A[7]	8	Mds[1][1]

Resulting Mds tile (shared by the whole block, reused for BOTH strips below):

	col 0	col 1
row 0	3	4
row 1	7	8

Strip c = 0 (col = colStart + 0*2)

Nds load — inside the c loop, fresh load. Corner-turned: $Nds[tx][ty] = B[loadCol*N + (1*2 + tx)]$, where $loadCol = bx*4 + 0*2 + ty$ (ty, not tx — the load column must match the slot's second index):

thread	loadCol	global read	value	written to
(0,0)	0	B[2]	3	Nds[0][0]
(1,0)	0	B[3]	4	Nds[1][0]
(0,1)	1	B[6]	7	Nds[0][1]
(1,1)	1	B[7]	8	Nds[1][1]

Nds buffer now holds:

	col 0	col 1
row 0	3	7
row 1	4	8

__syncthreads() — every load above must land before any thread computes.

Compute — each thread updates ONLY Pvalue[0]: for k in 0..1, $Pvalue[0] += Mds[ty][k] * Nds[k][tx]$

thread	terms (k=0 + k=1)	adds	Pvalue[0] running total
(0,0)	$3 \times 3 + 4 \times 4$	25	30
(1,0)	$3 \times 7 + 4 \times 8$	53	70
(0,1)	$7 \times 3 + 8 \times 4$	53	70
(1,1)	$7 \times 7 + 8 \times 8$	113	174

__syncthreads() — every thread must finish READING Nds before the next strip (or next phase) overwrites it.

Strip c = 1 (col = colStart + 1*2)

Nds load — inside the c loop, OVERWRITES the c=0 contents — same buffer, new data. Corner-turned: $Nds[tx][ty] = B[loadCol*N + (1*2 + tx)]$, where $loadCol = bx*4 + 1*2 + ty$ (ty, not tx — the load column must match the slot's second index):

thread	loadCol	global read	value	written to
(0,0)	2	B[10]	11	Nds[0][0]
(1,0)	2	B[11]	12	Nds[1][0]
(0,1)	3	B[14]	15	Nds[0][1]
(1,1)	3	B[15]	16	Nds[1][1]

Nds buffer now holds:

	col 0	col 1
row 0	11	15
row 1	12	16

__syncthreads() — every load above must land before any thread computes.

Compute — each thread updates ONLY Pvalue[1]: for k in 0..1, $Pvalue[1] += Mds[ty][k] * Nds[k][tx]$

thread	terms (k=0 + k=1)	adds	Pvalue[1] running total
(0,0)	$3 \times 11 + 4 \times 12$	81	110
(1,0)	$3 \times 15 + 4 \times 16$	109	150
(0,1)	$7 \times 11 + 8 \times 12$	173	278
(1,1)	$7 \times 15 + 8 \times 16$	233	382

__syncthreads() — every thread must finish READING Nds before the next strip (or next phase) overwrites it.

Final write — delivery from private registers to global P

thread	c	col = colStart+c*2	P address (row*N+col)	value written
(0,0)	0	0	$P[0] = P[0][0]$	30
(0,0)	1	2	$P[2] = P[0][2]$	110
(1,0)	0	1	$P[1] = P[0][1]$	70
(1,0)	1	3	$P[3] = P[0][3]$	150
(0,1)	0	0	$P[4] = P[1][0]$	70
(0,1)	1	2	$P[6] = P[1][2]$	278
(1,1)	0	1	$P[5] = P[1][1]$	174
(1,1)	1	3	$P[7] = P[1][3]$	382

No syncthreads needed here: each thread writes only its own P locations — no cooperation, no shared buffer.

Block (0,1) — output rows 2 and 3, all columns

Phase ph = 0

Mds load — happens ONCE this phase, outside the c loop. Each thread fetches $A[\text{row}][0..1 \text{ range}]$ via $A[\text{row} * N + (0 * 2 + \text{tx})]$:

thread	global read	value	written to
(0,0)	A[8]	9	Mds[0][0]
(1,0)	A[9]	10	Mds[0][1]
(0,1)	A[12]	13	Mds[1][0]
(1,1)	A[13]	14	Mds[1][1]

Resulting Mds tile (shared by the whole block, reused for BOTH strips below):

	col 0	col 1
row 0	9	10
row 1	13	14

Strip c = 0 (col = colStart + 0*2)

Nds load — inside the c loop, fresh load. Corner-turned: $Nds[\text{tx}][\text{ty}] = B[\text{loadCol} * N + (0 * 2 + \text{tx})]$, where $\text{loadCol} = \text{bx} * 4 + 0 * 2 + \text{ty}$ (ty, not tx — the load column must match the slot's second index):

thread	loadCol	global read	value	written to
(0,0)	0	B[0]	1	Nds[0][0]
(1,0)	0	B[1]	2	Nds[1][0]
(0,1)	1	B[4]	5	Nds[0][1]
(1,1)	1	B[5]	6	Nds[1][1]

Nds buffer now holds:

	col 0	col 1
row 0	1	5
row 1	2	6

__syncthreads() — every load above must land before any thread computes.

Compute — each thread updates ONLY Pvalue[0]: for k in 0..1, $Pvalue[0] += Mds[\text{ty}][k] * Nds[k][\text{tx}]$

thread	terms (k=0 + k=1)	adds	Pvalue[0] running total
(0,0)	$9 \times 1 + 10 \times 2$	29	29
(1,0)	$9 \times 5 + 10 \times 6$	105	105
(0,1)	$13 \times 1 + 14 \times 2$	41	41
(1,1)	$13 \times 5 + 14 \times 6$	149	149

__syncthreads() — every thread must finish READING Nds before the next strip (or next phase) overwrites it.

Strip c = 1 (col = colStart + 1*2)

Nds load — inside the c loop, OVERWRITES the c=0 contents — same buffer, new data. Corner-turned: $Nds[tx][ty] = B[loadCol*N + (0*2 + tx)]$, where $loadCol = bx*4 + 1*2 + ty$ (ty, not tx — the load column must match the slot's second index):

thread	loadCol	global read	value	written to
(0,0)	2	B[8]	9	Nds[0][0]
(1,0)	2	B[9]	10	Nds[1][0]
(0,1)	3	B[12]	13	Nds[0][1]
(1,1)	3	B[13]	14	Nds[1][1]

Nds buffer now holds:

	col 0	col 1
row 0	9	13
row 1	10	14

__syncthreads() — every load above must land before any thread computes.

Compute — each thread updates ONLY Pvalue[1]: for k in 0..1, $Pvalue[1] += Mds[ty][k] * Nds[k][tx]$

thread	terms (k=0 + k=1)	adds	Pvalue[1] running total
(0,0)	$9 \times 9 + 10 \times 10$	181	181
(1,0)	$9 \times 13 + 10 \times 14$	257	257
(0,1)	$13 \times 9 + 14 \times 10$	257	257
(1,1)	$13 \times 13 + 14 \times 14$	365	365

__syncthreads() — every thread must finish READING Nds before the next strip (or next phase) overwrites it.

Phase ph = 1

Mds load — happens ONCE this phase, outside the c loop. Each thread fetches $A[row][2..3 \text{ range}]$ via $A[row*N + (1*2 + tx)]$:

thread	global read	value	written to
(0,0)	A[10]	11	Mds[0][0]
(1,0)	A[11]	12	Mds[0][1]
(0,1)	A[14]	15	Mds[1][0]
(1,1)	A[15]	16	Mds[1][1]

Resulting Mds tile (shared by the whole block, reused for BOTH strips below):

	col 0	col 1
row 0	11	12
row 1	15	16

Strip c = 0 (col = colStart + 0*2)

Nds load — inside the c loop, fresh load. Corner-turned: $Nds[tx][ty] = B[loadCol*N + (1*2 + tx)]$, where $loadCol = bx*4 + 0*2 + ty$ (ty, not tx — the load column must match the slot's second index):

thread	loadCol	global read	value	written to
(0,0)	0	B[2]	3	Nds[0][0]
(1,0)	0	B[3]	4	Nds[1][0]
(0,1)	1	B[6]	7	Nds[0][1]
(1,1)	1	B[7]	8	Nds[1][1]

Nds buffer now holds:

	col 0	col 1
row 0	3	7
row 1	4	8

__syncthreads() — every load above must land before any thread computes.

Compute — each thread updates ONLY Pvalue[0]: for k in 0..1, $Pvalue[0] += Mds[ty][k] * Nds[k][tx]$

thread	terms (k=0 + k=1)	adds	Pvalue[0] running total
(0,0)	$11 \times 3 + 12 \times 4$	81	110
(1,0)	$11 \times 7 + 12 \times 8$	173	278
(0,1)	$15 \times 3 + 16 \times 4$	109	150
(1,1)	$15 \times 7 + 16 \times 8$	233	382

__syncthreads() — every thread must finish READING Nds before the next strip (or next phase) overwrites it.

Strip c = 1 (col = colStart + 1*2)

Nds load — inside the c loop, OVERWRITES the c=0 contents — same buffer, new data. Corner-turned: $Nds[tx][ty] = B[loadCol*N + (1*2 + tx)]$, where $loadCol = bx*4 + 1*2 + ty$ (ty, not tx — the load column must match the slot's second index):

thread	loadCol	global read	value	written to
(0,0)	2	B[10]	11	Nds[0][0]
(1,0)	2	B[11]	12	Nds[1][0]
(0,1)	3	B[14]	15	Nds[0][1]
(1,1)	3	B[15]	16	Nds[1][1]

Nds buffer now holds:

	col 0	col 1
row 0	11	15
row 1	12	16

__syncthreads() — every load above must land before any thread computes.

Compute — each thread updates ONLY Pvalue[1]: for k in 0..1, $Pvalue[1] += Mds[ty][k] * Nds[k][tx]$

thread	terms (k=0 + k=1)	adds	Pvalue[1] running total
(0,0)	$11 \times 11 + 12 \times 12$	265	446
(1,0)	$11 \times 15 + 12 \times 16$	357	614
(0,1)	$15 \times 11 + 16 \times 12$	357	614
(1,1)	$15 \times 15 + 16 \times 16$	481	846

__syncthreads() — every thread must finish READING Nds before the next strip (or next phase) overwrites it.

Final write — delivery from private registers to global P

thread	c	col = colStart+c*2	P address (row*N+col)	value written
(0,0)	0	0	$P[8] = P[2][0]$	110
(0,0)	1	2	$P[10] = P[2][2]$	446
(1,0)	0	1	$P[9] = P[2][1]$	278
(1,0)	1	3	$P[11] = P[2][3]$	614
(0,1)	0	0	$P[12] = P[3][0]$	150
(0,1)	1	2	$P[14] = P[3][2]$	614
(1,1)	0	1	$P[13] = P[3][1]$	382
(1,1)	1	3	$P[15] = P[3][3]$	846

No syncthreads needed here: each thread writes only its own P locations — no cooperation, no shared buffer.

Cross-check: kernel result vs hand-computed A x B

P assembled from all 16 writes

	col 0	col 1	col 2	col 3
row 0	30	70	110	150
row 1	70	174	278	382
row 2	110	278	446	614
row 3	150	382	614	846

Reference: A x B by ordinary matrix math

	col 0	col 1	col 2	col 3
row 0	30	70	110	150
row 1	70	174	278	382
row 2	110	278	446	614
row 3	150	382	614	846

MATCH — all 16 cells agree. The coarsened kernel is verified.

How to spot-check one cell by hand (recommended):

Pick $P[0][2] = 110$. By definition it is $A[0][0]*B[0][2] + A[0][1]*B[1][2] + A[0][2]*B[2][2] + A[0][3]*B[3][2] = 1 \times 9 + 2 \times 10 + 3 \times 11 + 4 \times 12 = 9 + 20 + 33 + 48 = 110$. In the trace, this cell was built by Block(0,0) thread (0,0) in its Pvalue[1] slot: phase 0 / c=1 contributed the first two terms, phase 1 / c=1 the last two. Follow that one thread through the tables above and watch the running total hit the same number.

The three structural facts to see in the tables:

1. Each phase has ONE teal Mds table but TWO coral Nds tables — the load asymmetry, visible. The same Mds values appear in both strips' compute terms (left factor), while the Nds values change per strip (right factor).
2. The two coral Nds tables per phase are the SAME buffer at two moments — the second overwrites the first. That is why the second sync exists.
3. Each thread's Pvalue[0] and Pvalue[1] columns accumulate across phases and never mix — phase 1's running totals continue from phase 0's, and c=0 computes never touch Pvalue[1].